

are there any nlp libraries built on rust or julia?

Yes, both Rust and Julia have dedicated natural language processing (NLP) libraries, though they serve slightly different purposes in the development ecosystem compared to Python's long-standing dominance. 🗳️ Reddit · r/rust +4

Rust focuses heavily on lightning-fast tokenization, deterministic safety, and highly optimized large language model (LLM) inference. Julia, on the other hand, excels in scientific computing, native array manipulations, and dynamic, mathematical text exploration. 🗳️ Reddit · r/rust +5

Notable Rust NLP Libraries

Rust's ecosystem is heavily anchored around performance, memory safety, and production-ready machine learning components. 🦀 rustify.rs +4

- **tokenizers** (by Hugging Face): The gold standard for modern NLP tokenization. This library provides ultra-fast implementations of modern tokenizers like Byte-Pair Encoding (BPE), WordPiece, and Unigram. It provides the performance backbone for most Python transformer pipelines via Rust bindings. 🗳️ Reddit · r/rust +3
- **candle** (by Hugging Face): A minimalist, pure-Rust machine learning framework. It acts as a lightweight alternative to PyTorch, specifically optimized to build, fine-tune, and run deep learning NLP models (like LLaMA, BERT, and Whisper) with zero Python overhead. 🐙 GitHub +4
- **mistral.rs**: A highly optimized, blazingly fast framework for local LLM inference. It supports multi-architectures, tokenization, and quantisation out of the box. 🐙 GitHub +4
- **lingua-rs**: The most accurate natural language detection library available for Rust, explicitly built for long and short text snippets alike. 🐙 GitHub... +1
- **natural**: A traditional, pure-Rust utility library. It handles fundamental tokenization, n-grams, phonetics (Soundex), and basic Naive Bayes text classification. 📺 YouTube · JCharisTech +1

Notable Julia NLP Libraries

Julia's NLP landscape leans heavily into mathematical text representation and integrated machine-learning pipelines.  Analytics Vidhya +2

- [TextAnalysis.jl](#): The core, most feature-complete suite for traditional NLP in Julia. It manages text preprocessing, creating Document-Term Matrices (DTMs), tf-idf calculations, Latent Semantic Analysis (LSA), and Latent Dirichlet Allocation (LDA).  juliapackages.com +1
- [WordTokenizers.jl](#): A high-performance package dedicated solely to string tokenization and sentence splitting. It forms the underlying structural foundation for other complex text toolkits in Julia.  juliapackages.com
- [Embeddings.jl](#): A streamlined package providing instant, native access to pre-trained word representations like Word2Vec, FastText, and GloVe vectors.  juliapackages.com +2
- [Flux.jl](#) / [Lux.jl](#): While these are general-purpose neural network frameworks, they are frequently utilized alongside [TextAnalysis.jl](#) to build custom deep-learning recurrent architectures or basic transformer models without leaving native Julia syntax.  The Julia Programming Language +2

Summary Comparison

Feature / Goal  Medium · Matt Welsh +11			Rust Ecosystem	Julia Ecosystem
Primary Strength			Speed, memory efficiency, deployment binaries.	Rapid math prototyping, data science workflows.
Best Used For			Fast tokenization, local LLM deployment, multi-threading.	Statistical NLP, text mining, vector-space mathematics.
Ecosystem Style			Modular, low-level building blocks.	Integrated, sci-comp and math-native packages.

Are you looking to build a **classical NLP pipeline** (like regex tokenization, stemming, or tf-idf), or are you trying to deploy and run **modern LLM-based architectures**? Knowing your specific goal will help determine whether Rust or Julia fits your stack better!  YouTube · Tsoding Daily +4



 41 sites

Rust for machine learning advantages! - Reddit

21 Feb 2024 — Huggingface, a well-known NLP library, releases tokenizers in Rust for order of magnitude speed improvement.

 Reddit · r/rust

Natural Language Processing in Rust - Reddit

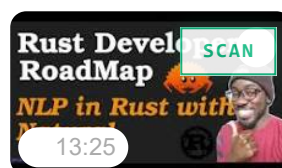
21 Dec 2016 — Comments Section. PXaZ. • 10y ago. writing one from scratch in Rust can be really challenging and fun. If I'm not mistaken, gcollu...

 Reddit · r/rust

Natural Language Processing in Rust with Natural.rs

24 Jan 2025 — all right so welcome back so in the previous tutorial we saw how to do sentiment analysis in Rust which is still part of the NL...

 YouTube · JCharisTech



Show all

How do these libraries compare with spacy and spark-nlp?

spaCy and **Spark NLP** are monolithic, enterprise-ready frameworks designed to handle complete, industrial-strength NLP pipelines out of the box.  Medium · Mustafa Aytuğ Kaya +1

By contrast, **Rust and Julia NLP libraries are modular building blocks**. They do not yet offer the same all-in-one maturity, but they aggressively outperform spaCy and Spark NLP in narrow, highly specific niches like raw execution speed, memory footprint, and custom mathematical architectures.  Reddit · r/rust +1

The Main Ecosystem Difference

- **spaCy (Python/Cython):** Optimized for **single-machine developer ergonomics**. It provides pre-trained, production-ready neural pipelines

(parsing, NER, lemmatisation) that work instantly with one line of code.  LinkedIn · James Hall +1

- **Spark NLP (Java/Scala/Python):** Optimized for **distributed big data clusters**. It is built directly on top of Apache Spark to process terabytes of text across hundreds of machine nodes simultaneously.  O'Reilly Media +1
- **Rust NLP:** Built for **zero-overhead deployment and micro-optimisation**. It strips away runtime environments (like Python or the Java Virtual Machine) to run inference directly on bare metal or edge devices.  Reddit · r/rust +1
- **Julia NLP:** Built for **mathematical and statistical research**. It treats text as native matrix/array operations, bypassing the "two-language problem" (where you prototype in Python but deploy in C++).  Reddit · r/rust +1

Direct Comparison

Feature  LinkedIn · James Hall +6	spaCy	Spark NLP	Rust Libraries (e.g., <code>candle</code> , <code>tokenizers</code>)
Architectural Focus	Local execution, ease of use.	Horizontal scalability across clusters.	Extreme memory efficiency for local LLMs.
Batteries Included?	Yes. Tokenizer, POS, NER, and Linker in one object.	Yes. Massive library of enterprise clinical/legal pipelines.	No. You must stitch tokenizers, tensors, a weights together.
Scaling Capability	Limited to single-node RAM/CPU limits.	Infinite (scales seamlessly to multi-terabyte data).	Vertical scaling (superthreading via Rayon).
Memory Footprint	Moderate (heavy model weight dependency).	Extremely High (requires a heavy JVM cluster infrastructure).	Extremely Low. Ideal for binary distribution & VLMs.
Language Support	Excellent (dozens of languages)	Excellent (massive language support)	Dependent on HuggingFace hub configurations.

robust, pre-
trained
languages).
multilingual
support).

Head-to-Head Breakdown

1. Rust vs. spaCy (The Local Speed Battle)

While spaCy is fast because it is written in Cython, it still carries Python baggage. Rust's `tokenizers` library is so much faster than spaCy's native tokenization that **Hugging Face actually wrote `tokenizers` in Rust to back their own Python library**. However, if you need Named Entity Recognition (NER), spaCy provides a production-ready model instantly. In Rust, you would have to download raw model weights, use `candle` to construct the neural network layers manually, and handle the tensor shapes yourself.  LinkedIn · James Hall +2

2. Rust vs. Spark NLP (The Infrastructure Battle)

Spark NLP requires an active Apache Spark cluster, meaning Java Virtual Machine (JVM) overhead, complex configuration, and massive memory usage. If you have 50 terabytes of logs on a cluster, Spark NLP wins hands-down. But if you want to run high-throughput text processing on a single 32-core server, a Rust-based pipeline will routinely match or beat Spark NLP's throughput using a fraction of the RAM, completely eliminating the need for a distributed cluster.  www.oreilly.com

3. Julia vs. spaCy / Spark NLP (The Prototyping Battle)

If you want to modify how a Transformer or an embedding algorithm works mathematically at a foundational level, doing it in spaCy or Spark NLP is incredibly difficult because their core engines are locked behind compiled Cython or Java wrappers. Julia allows you to write custom linear algebra formulas directly over text document-term matrices at native C-speeds, offering a level of algorithmic flexibility that the enterprise giants cannot match.  LinkedIn · James Hall +1

If you are building a standard enterprise application, **spaCy** (for single servers) or [Spark NLP](#) (for massive data lakes) remain the practical choices. You should only swap them out for **Rust** if you have strict hardware/memory limits (like

edge computing), or **Julia** if your project requires custom scientific or matrix-heavy text manipulation.  LinkedIn · James Hall +4

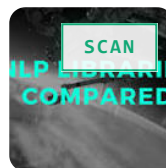
What does your current infrastructure look like? If you share whether you are deploying to **cloud microservices**, **a big data cluster**, or **local machines**, I can point you toward the most efficient stack.



 8 sites

NLP Libraries Compared - LinkedIn

2 Apr 2020 — SpaCy is a popular library for NLP for several key reasons. It is one of the fastest of the libraries compared here, in large part...



LinkedIn · James Hall

I'm considering Rust, Go, or Julia for my next language and I'd ...

17 Apr 2021 — For data science, I'd suggest Julia. That's its focus and it's really good at it. At runtime, it'll be almost as fast as Rust, an...



Reddit · r/rust

SpaCy or Spark NLP — A Benchmarking Comparison - Medium

26 Aug 2020 — Comparing two different libraries is not as simple as it sounds. Each library has different implementation methods and thus will h...



Medium · Mustafa Aytuğ Kaya

Show all

So if I have a pipeline like for the following projects, could I start with either spacy/spark-nlp, then incrementally add in the higher performing rust/julia libraries?

<https://huggingface.co/datasets/edithatogo/nz-hansard-corpus>

<https://github.com/edithatogo/corpus-nz-hansard>

<https://huggingface.co/datasets/edithatogo/nz-hansard-source-archive>

<https://huggingface.co/datasets/edithatogo/nz-legislation-corpus>



For processing the NZ Hansard and Legislation datasets, a hybrid approach starting with Rust-backed Python tools like Polars and spaCy is recommended over Spark, which is over-engineered for this data scale. This strategy provides an efficient, incremental path to high performance, allowing for the later integration of Rust for bottlenecks or Julia for complex network analysis.